

Code Explanation

DLT Pipeline Creation and Execution Code Explanation

The provided code is used to create and run a Delta Live Tables (DLT) pipeline using the Databricks Jobs API. It defines a function `create\_and\_run\_pipeline` that creates or updates a DLT pipeline based on a predefined configuration and starts a pipeline update.

Overview

This code is designed to be used in a Databricks notebook and utilizes the `%run` magic command to execute the pipeline. It handles the creation or update of a DLT pipeline and initiates a pipeline update.

Step 1: Importing Necessary Modules and Initializing Workspace Client

This step involves importing the required modules from the `databricks.sdk` library and initializing a `WorkspaceClient` object to interact with the Databricks workspace.

```
from databricks.sdk import WorkspaceClient
from databricks.sdk.service import jobs

# Initialize the Workspace client
ws = WorkspaceClient()
```

Step 2: Defining the Pipeline Configuration

In this step, the pipeline configuration is defined, including the pipeline name, cluster settings, development mode, and library dependencies.

```
pipeline_name = "Customer Order Data Pipeline"

pipeline_config = {
    "name": pipeline_name,
    "clusters": [
        {
            "label": "default",
            "num_workers": 2,
            "autoscale": {
                "min_workers": 1,
                "max_workers": 4
            }
        }
    ],
    "development": True,
    "continuous": False,
    "libraries": [
        {
            "notebook": {
                "path": "/Repos/your_repo_path/src/dlt_pipeline"
            }
        }
    ],
    "target": f"{CATALOG}.{SCHEMA}",
    "configuration": {
        "pipelines.enableTrackHistory": "true"
    }
}
```

Step 3: Creating or Updating the Pipeline

This step checks if a pipeline with the specified name already exists. If it does, the existing pipeline is updated; otherwise, a new pipeline is created.

```
try:
    existing_pipelines = ws.pipelines.list(filter=f"name='{pipeline_name}'")
    pipeline_id = None

    for pipeline in existing_pipelines:
        if pipeline.name == pipeline_name:
            pipeline_id = pipeline.pipeline_id
            break

    if pipeline_id:
        # Update existing pipeline
        ws.pipelines.edit(pipeline_id=pipeline_id, **pipeline_config)
        print(f"Updated existing pipeline with ID: {pipeline_id}")
    else:
        # Create new pipeline
        response = ws.pipelines.create(**pipeline_config)
        pipeline_id = response.pipeline_id
        print(f"Created new pipeline with ID: {pipeline_id}")
```

Step 4: Starting a Pipeline Update

After creating or updating the pipeline, this step initiates a pipeline update using the `start\_update` method.

```
# Start a pipeline update
ws.pipelines.start_update(pipeline_id=pipeline_id)
print(f"Started pipeline update for pipeline ID: {pipeline_id}")
```

Step 5: Handling Exceptions

This step catches any exceptions that occur during the pipeline creation or update process and prints the error message.

```
except Exception as e:
    print(f"Error creating or running pipeline: {str(e)}")
```

Step 6: Running the Function to Create and Start the Pipeline

Finally, the `create\_and\_run\_pipeline` function is called to execute the pipeline creation and update process.

```
# Run the function to create and start the pipeline
create_and_run_pipeline()
```